SYNTHOSEC TECHNICAL REPORT

Synthetic Cybersecurity Log Platform Architecture & Implementation Review

PROJECT REPOSITORY
https://github.com/riagupta240619/Synthetic-Log-generator

PREPARED DATE
20 October 2026

DOCUMENT TYPE
Technical Project Report & Implementation Review

PRIMARY FOCUS
Architecture, Data Flow, Generation & SIEM Validation

EXECUTIVE SUMMARY

SynthoSec is a full-stack platform engineered to produce synthetic cybersecurity telemetry and leverage that data to test log-quality checks and SIEM-oriented detection logic. The platform combines a React + TypeScript web interface with a Python FastAPI backend service. Generation methods include hand-authored scenario logic, file-pattern learning, cloud audit event simulation, natural-language prompting with local Ollama support, and Markov-chain sequence modeling. Generated records are structured into datasets, validated for consistency, and exported as JSON, NDJSON, CSV, or BSD/IETF Syslog. A Wazuh-oriented matcher evaluates event text against rule signatures and maps detected alerts to MITRE ATT&CK tactics.

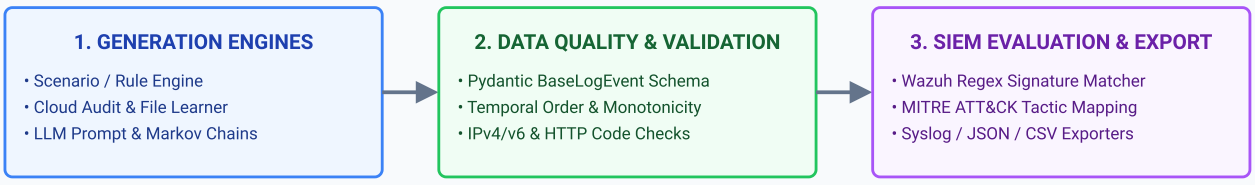
1. INTRODUCTION

1.1 Background: Security operations center (SOC) environments rely heavily on structured log telemetry from endpoints, servers, firewalls, and cloud infrastructure. Sharing real telemetry poses significant privacy and compliance challenges, while edge-case attack events are often absent from standard baseline logs. Synthetic logs resolve this by providing repeatable, customizable, and labeled datasets for parser validation, dashboard building, and SIEM rule testing.

1.2 Problem Statement: The central engineering challenge is generating coherent, schema-compliant security events at configurable volume while preserving realistic event relationships, sequences, and threat indicators without relying on sensitive operational data.

1.3 Objectives: Produce multi-domain events (Linux, Windows, Web, Cloud Audit); provide diverse generation strategies; infer field distributions from uploaded samples; perform integrity validation; export to standard SIEM formats; and match events against SIEM rulesets.

PLATFORM FUNCTIONAL ARCHITECTURE



2. SCOPE AND INTENDED USE CASES

Included Scope: Multi-vector attack scenario generation (SSH brute-force, privilege escalation, web exploits, ransomware staging), Cloud audit log synthesis (AWS CloudTrail, Azure Activity, GCP Audit), statistical file learning, prompt-driven LLM event generation, stateful Markov sequences, automated schema validation, and Wazuh integration.

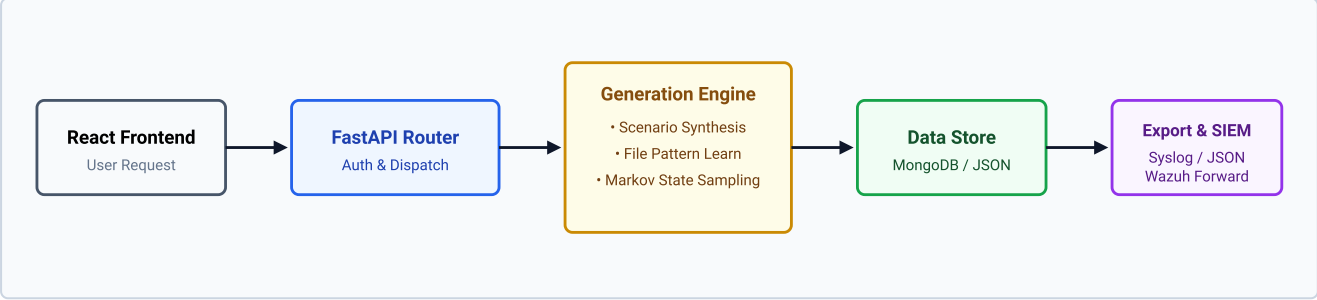
Intended Uses: Defensive SIEM testing in isolated labs, SOC analyst training, parser and rule development, dataset generation, and pipeline benchmarking. Synthetic logs should be clearly marked as synthetic.

3. SYSTEM ARCHITECTURE

The platform uses a decoupled frontend-API-engine model. The browser frontend communicates with a Python FastAPI service backed by MongoDB (with a transparent local JSON file fallback for offline/development execution).

Layer	Technologies / Modules	Primary Responsibility
Presentation	React 19, TypeScript, Vite, Lucide	User workflows, dataset controls, real-time validation dashboards, Wazuh evaluation views.
API Layer	FastAPI, Uvicorn, Pydantic	REST endpoints, payload parsing, authorization headers (X-API-Key), execution dispatch.
Engines	Scenario, Cloud, FileLearner, LLM, Markov	Event synthesis logic, template expansion, state transitions, distribution sampling.
Quality Assurance	BaseLogEvent, LogValidator	Schema validation, IP syntax checking, timestamp verification, integrity score calculation.
Persistence	MongoDB (Motor driver), Local JSON	Dataset storage, metadata indexing, validation report caching, export staging.
SIEM Integration	WazuhRulesetMatcher, Forwarder	Regex signature matching, severity scoring, ATT&CK mapping, UDP/TCP forwarding.

Data Flow Pipeline

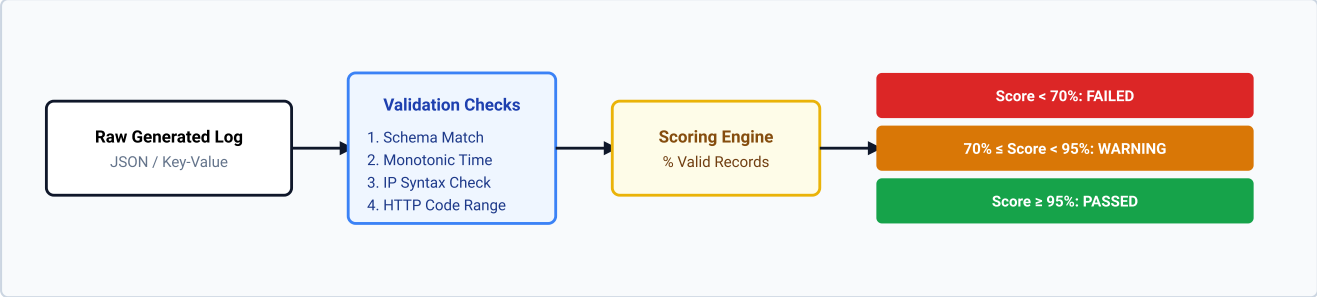


4. GENERATION ENGINE CAPABILITIES

Engine	Technique & Input	Strengths	Considerations
Scenario Logic	Hand-crafted attack narratives with labeled anomalies and noise events.	Deterministic ground-truth labels and sequence tracking.	Fixed patterns can become predictable over time.
Cloud Audit	Provider-specific templates (AWS CloudTrail, Azure, GCP).	Realistic cloud control-plane telemetry structure.	Simulated API structures require periodic schema updates.
File Learner	Pandas profiling of uploaded CSV/JSONL sample logs.	Preserves field distributions and categorical values.	Requires statistical verification to ensure zero re-identification.
LLM Prompt	Natural-language prompting via Ollama with keyword fallback.	Highly dynamic, flexible prompt-driven generation.	Fallback uses static keyword-matching templates.
Markov Chain	Stochastic transition matrix across attack stages.	Realistic inter-arrival times and state progression.	Default probabilities are pre-configured in source code.

5. VALIDATION FRAMEWORK & QUALITY SCORING

To ensure logs meet standard ingestion requirements for SIEMs, SynthoSec runs each record through an automated validation pipeline assessing four core operational dimensions:



6. WAZUH EVALUATION & FORWARDING

The system features an in-process regular expression matching engine (`WazuhRulesetMatcher`) that scans event messages against predefined security rules. Matches yield detection alerts tagged with severity levels and MITRE ATT&CK tactical mappings. An optional transport layer enables streaming logs directly to an external Wazuh manager via TCP or UDP syslog.

7. VERIFICATION STATUS & SOURCE CODE FINDINGS

> **Source Review Finding:** In `backend/tests/test_scenarios.py`, the unit test `test_validator_and_wazuh_ruleset` generates 40 events but explicitly asserts `report.total_logs == 20`. This assertion mismatch should be corrected to prevent automated build failures in clean CI pipelines.

Additional verification items include tightening the backend CORS policy (currently defaulted to `allow_origins=["*"]`), purging committed `__pycache__` artifacts, and implementing standard API rate-limiting.

8. RECOMMENDED ENGINEERING ROADMAP

Priority	Work Item	Target Acceptance Criteria
P0	Fix test suite assertion mismatches.	100% passing rate in clean Pytest execution.
P0	Harden API security & CORS configuration.	Explicit origin whitelist, strict API key rotation.
P1	Ground-truth precision/recall benchmark.	Automated detection rate scoring against known labels.
P1	Live Wazuh manager validation lab.	Comparative report between local regex & live decoders.
P1	Expand provider native schemas.	Full structural compliance for AWS, Azure, and GCP.
P2	Enhance file learner dependencies.	Conditional field correlation and inter-arrival time modeling.
P2	Add deterministic random seeding.	Reproducible dataset regeneration using explicit seeds.

9. CONCLUSION

SynthoSec provides an architecture for synthetic telemetry generation and SIEM evaluation. By integrating multiple synthesis engines behind a unified FastAPI and React application, it provides security teams with a tool for generating test telemetry. Implementing the P0/P1 roadmap items will further solidify its reliability for operational security testing.